

# Build Your Second AI Brain + Obsidian

OpenClaw Masterclass — Lesson Guide

---

## The big picture

Your agent already has a good base thanks to User.md, which covers the basics about you and your business. But most of your day-to-day work lives in small details that don't fit into one file — notes about clients, project updates, meeting notes, random ideas, research. This guide shows you how to set up an Obsidian vault that your agent can read and write to, so you both share the same notes that grow over time.

## What this guide covers

- Installing Obsidian and setting up a clean six-folder vault
- Creating a private GitHub repo that keeps your laptop and VPS in sync
- Turning on automatic two-way sync in Obsidian with the Git plugin
- Connecting your agent on the VPS to the same repo and saving credentials safely
- Giving your agent permanent rules so it always syncs before reading and after writing
- A bonus prompt that fills the vault with demo content for a nice graph view

## Before you start

- Your agent running on a Hostinger VPS, or on your own machine via the local install path
- Telegram or the OpenClaw dashboard open so you can send your agent messages
- A free GitHub account — you can sign up during the lesson if you don't have one
- About 20 minutes from start to finish

# The idea behind a second brain

Think of User.md as a short profile of who you are and how your business works. It loads into every conversation and gives your agent a basic understanding of your situation. A vault is something different — it's a growing library of specific notes. One note for each client, one for each project, notes for daily reflections, notes for research. Together, User.md tells your agent *who you are*, and the vault tells your agent *what you know*. Both are useful in their own way.

Obsidian is a free note-taking app. Every note is a plain markdown text file saved in a folder on your computer. There's no database, no special format, no cloud lock-in. That's what makes it a great fit for an AI agent — markdown is a format every language model already understands, so your agent can read, search, update, and create notes without any complex setup.

## How the pieces fit together

Obsidian runs on your laptop and your agent runs on a remote VPS, so the two machines can't see each other's files directly. We connect them with a private GitHub repository that both sides push to and pull from. Here's how it works:

- Obsidian on your laptop keeps your notes in a local folder
- The Obsidian Git plugin handles commit, push, and pull for you in the background
- GitHub holds the main copy that both machines sync with
- Your agent clones the same repo into its workspace on the VPS
- Before reading any note, your agent runs git pull to get your latest edits
- After creating or updating any note, your agent runs git commit and git push so your laptop sees it
- A five-minute interval keeps both sides in sync without hammering GitHub

**On the local install path?** You can follow a shorter setup because your agent and Obsidian live on the same computer. GitHub becomes optional in that case, but it's still worth adding later as a free backup and for syncing your vault to other devices. Jump to the **Local install path** section at the end for the details.

# Setup walkthrough

## Step 1 — Install Obsidian

Go to `obsidian.md`, click the download button for your operating system, and run the installer. The app is free on every platform and the install takes about a minute. When you open Obsidian for the first time, you'll see a welcome screen with a few options. Leave it on that screen for now — we'll come back to it after setting up GitHub first.

## Step 2 — Create a private GitHub repository

Open `github.com` in a new tab. Create a free account if you don't have one — you just need an email address. Once you're signed in, click the plus icon at the top right of the page and choose **New repository**. Fill in the form like this:

- **Repository name:** `ace-vault` — keep it lowercase with dashes
- **Description:** anything you want, like 'Shared vault for me and my agent'
- **Visibility:** set it to **Private** so your notes aren't public
- **Add a README file:** turn this toggle on
- **.gitignore / license:** leave them as None
- Click the green **Create repository** button at the bottom

**Why no spaces or special characters?** When your agent later reads files from the vault using shell commands, folder names with spaces or special characters can cause problems. Sticking to lowercase letters, numbers, and dashes keeps things simple.

## Step 3 — Clone the repo to your laptop

On the repository page, click the green **Code** button and switch to the **HTTPS** tab. Copy the URL shown there — it'll look like `https://github.com/your-username/ace-vault.git`.

Now open a terminal. On Mac press `Cmd+Space` and type `terminal`. On Windows search for PowerShell in the start menu. Then run these commands, replacing the URL with the one you just copied:

```
cd ~/Documents
git clone https://github.com/your-username/ace-vault.git
ls
```

If it's the first time git is running on your computer, a browser window might pop up asking you to sign in to GitHub — just go through it and come back to the terminal. When the clone finishes, running `ls` should show an `ace-vault` folder inside Documents.

**Cloud sync warning:** if your Documents folder on Mac is synced to iCloud Drive (look for small cloud icons in Finder), iCloud can cause problems with git. Either pick a location outside iCloud, or turn off iCloud sync for Documents in System Settings. Windows users have the same issue with OneDrive — pause OneDrive for the vault folder or move the vault somewhere else.

## Step 4 — Open the cloned folder as a vault

Switch back to Obsidian on the welcome screen. Click **Open** next to **Open folder as vault**, find **Documents** → **ace-vault**, select it, and click Open. Obsidian might ask you to confirm that you trust this vault — since you just made it yourself, click **Trust author and enable plugins**. The vault opens with a single README file, which is the one GitHub added when you created the repository.

## Step 5 — Create the folder structure

Six folders give you a solid starting point that covers most of what comes up in daily work. Create them from the file explorer by clicking the new folder icon or right-clicking in the empty space. Here's what each one is for:

| Folder    | Purpose                                                               |
|-----------|-----------------------------------------------------------------------|
| Inbox     | Quick notes and unsorted thoughts that you will sort later            |
| Daily     | Daily notes for reflections, logs, and journal entries tied to a date |
| Projects  | One note or subfolder for each active project you are working on      |
| Resources | Reference material, research, articles, and useful snippets           |
| Business  | Your offers, ICP, pricing, and anything about your business itself    |
| People    | Notes about clients, collaborators, and people in your network        |

## Step 6 — Add a first note

Right-click the **Business** folder and choose **New note**. As soon as the input field appears, type **about-my-business** and press Enter to confirm the name. If you click into the editor before pressing Enter, Obsidian sometimes loses the name and creates an Untitled note in the wrong place. Name first, Enter second, edit third.

Click into the editor on the right and write a few simple sentences about what your business does — your customers, your offers, what problems you solve. Here's an example you can adapt to your own situation:

*My business builds AI solutions for SMEs and helps them automate their processes. My main customers are small and medium business owners who want to save time on repetitive work. I help them set up AI agents that handle tasks like email management, content creation, customer follow-ups, and data organisation.*

## Step 7 — Install the Git plugin

Click the gear icon in the bottom-left of Obsidian to open Settings. In the left sidebar, click **Community plugins**. If you've never used community plugins before, Obsidian will show a warning about third-party plugins — click **Turn on community plugins** to continue. The plugin we're about to install has millions of users and is well maintained, so it's safe.

Click **Browse** to open the plugin marketplace. Type **git** in the search field. The plugin you want is called **Git** by **Vinzent** — it's near the top with a huge download count. Click it, then click **Install**. After it installs, the button changes to **Enable**. Click that to turn it on in your vault.

**Heads up:** there's another plugin called **GitHobs** that sometimes shows up in the search results. That's not the right one. Make sure you install **Git** by Vinzent specifically.

## Step 8 — Turn on automatic sync

Close the marketplace. Back in Settings, the left sidebar now shows a new entry for **Git** under Community plugins. Click it to open the plugin settings. Scroll down until you find the **Automatic** section — this is where we set up the sync.

| Setting                                    | Value                    |
|--------------------------------------------|--------------------------|
| Split timers for automatic commit and sync | Toggle ON (turns purple) |
| Auto commit interval (minutes)             | 5                        |
| Auto push interval (minutes)               | 5                        |
| Auto pull interval (minutes)               | 5                        |

These four values make the plugin run in the background. Every five minutes it commits your changes, pushes them to GitHub, and pulls down anything new from the other side. That's what keeps your laptop and your agent's workspace in sync.

## Step 9 — Trigger a manual push to check it works

Open the command palette with **Cmd+P** on Mac or **Ctrl+P** on Windows. Type **commit-and-sync** and pick **Git: Commit-and-sync** from the list. A success notification pops up in the top right corner when the push finishes. Switch to your GitHub tab and refresh the repository page — the **Business** folder and your about-my-business note should now be visible. The other five folders won't show up yet because git only tracks folders that have at least one file in them.

# Connecting your agent to the vault

Everything so far has been on the laptop side. Now we set up the VPS side so your agent can read and write the same vault. Open your chat with your agent on Telegram or the dashboard and work through the prompts below in order. Each prompt builds on the one before it.

## Prompt 1 — First clone attempt (it will fail on purpose)

This first prompt asks your agent to try cloning the repo without any credentials. Because the repo is private, GitHub will block it and your agent will come back asking for a token. That's exactly what we want — it makes the next step feel natural. Swap in your own GitHub username before sending:

*I want you to clone my private GitHub repo into your workspace at ~/ace-vault. The repo URL is <https://github.com/your-username/ace-vault.git>. The repo is private so you will need a Personal Access Token. Try cloning first — if it fails with authentication error, stop and tell me, I will create a token and give it to you. After cloning, just list the contents of the folder and tell me what you see.*

Your agent replies with a git error about not being able to read the username for github.com. That's the signal to create a token.

## Create a Personal Access Token

Go to <https://github.com/settings/tokens> in a new tab.

- Click **Generate new token** → **Generate new token (classic)**
- In the **Note** field, type `ace-vault-access`
- Set **Expiration** to 90 days — or longer if you prefer
- Under **Select scopes**, tick just the **repo** checkbox (the first big one)
- Don't tick anything else — extra permissions make the token more risky if it leaks
- Scroll down and click the green **Generate token** button
- Copy the token straight away — it starts with `ghp_`

**Treat this token like a password.** GitHub only shows it to you once — as soon as you leave the page, the token is gone for good. If you're recording a video of this process, plan to delete this token afterwards and create a new one for real use. Never paste tokens into public places or commits.

## Prompt 2 — Send the token and clone

Now send your agent the token along with the exact command to use. Replace **ghp\_yourtokenhere** with your actual token and **your-username** with your GitHub username:

*Here is the Personal Access Token: ghp\_yourtokenhere*

*Use this token to clone the repo. The git command should be:*

*git clone https://ghp\_yourtokenhere@github.com/your-username/ace-vault.git ~/ace-vault*

*After cloning, list the contents of the folder and tell me what you see.*

Putting the token inside the URL between **https://** and **@github.com** is the standard way to clone a private repo with a PAT. After a few seconds your agent reports what's inside the folder: **.git**, **.obsidian**, **Business**, and **README.md**.

## Prompt 3 — Save the credentials safely

Your agent might refuse to save the token to disk by default, which is actually good behaviour from a security point of view. But that's a problem for us because every future push or pull would need the token again. The fix is to use git's credential helper in a way that only applies to this one repo, not the whole system:

*Good thinking on not persisting the token by default. Please save the token securely so future git operations work automatically. Use git's credential helper with the store mode but only inside the ace-vault folder, not globally. Run this command from inside ~/ace-vault: git config credential.helper store. Then make one commit and push so the credentials get cached for that repo only.*

## Prompt 4 — Give your agent a git identity

When your agent tries to make its first test commit, git refuses because there's no author name or email set for the repo. Send your agent the values it needs. You can put your agent's actual name in the **user.name** field — I'm using Ace in the example below, but use whatever you've named yours:

*Use these for the repo-local git config: git config user.name "Ace" and git config user.email "ace@yourdomain.com". Then finish the credential setup, do a small test commit and push, and confirm everything works.*

The name and email don't need to be real — they're just labels that git adds to each commit. Using your agent's name as the author makes it easy later to tell which commits came from the VPS and which came from your laptop.

## Prompt 5 — Give your agent the workflow rules

This prompt is the most important one in the lesson. It sets the rules your agent will follow every time it uses the vault, in every future conversation. The three rules cover pulling before reading, pushing after writing, and respecting the folder layout:

*From now on, follow these rules every single time you interact with my Obsidian vault at ~/ace-vault.*

*Rule one: BEFORE reading anything from the vault, run git pull inside the vault folder so you always have the latest version I might have edited on my laptop.*

*Rule two: AFTER creating or updating any note inside the vault, run git add, git commit with a meaningful message, and git push so my laptop receives your changes.*

*Rule three: the vault has six folders inside it: Inbox, Daily, Projects, Resources, Business, and People. Treat each folder according to its purpose.*

*Save these rules permanently to your memory so you remember them in every future conversation. When you're done, confirm you've saved the rules.*

Don't skip the confirmation step — make sure your agent actually says it has saved the rules to long-term memory. Without that, the rules only apply to this conversation and will be forgotten next time you talk to your agent.

## Testing the setup both ways

### Read test — your agent reads one of your notes

First check that your agent can read what you wrote on your laptop. Send this prompt:

*Run git pull inside the vault, then read the about-my-business note in the Business folder and tell me what's inside it.*

Your agent pulls the latest from GitHub, opens the file, and replies with the contents. Seeing your own words come back through the agent proves that the read side works end to end.

## Write test — your agent creates a new note you can see

Now test the other direction. Ask your agent to add a new file to the Projects folder and push it, then check if it shows up on your laptop:

*Now create a new note inside the Projects folder called Q1 Marketing Plan with three short bullet points: one about social media strategy, one about email campaigns, and one about content creation. After saving the file, commit and push so I can see it on my laptop.*

Once your agent confirms the commit is done, switch back to Obsidian. The new note won't appear right away because the auto-pull runs every five minutes. You have two choices: wait a few minutes for the scheduled pull, or trigger a pull right away using the command palette (**Cmd+P** or **Ctrl+P**, then type **git pull**). Either way, **Q1 Marketing Plan** shows up in the Projects folder with the three bullet points your agent wrote.

## Optional — Fill the vault with demo content

Once the sync is working, it's worth filling the vault with a realistic set of notes so the graph view has something to show. The prompt below asks your agent to create around 18 connected notes across all six folders in a couple of minutes. Feel free to change the names and topics to match your own business before sending:

I want you to populate my Obsidian vault with realistic demo content so the graph view looks rich and interconnected. Run git pull first to get the latest version. Then create the following notes with `[[wikilinks]]` between them so they form a connected graph.

In People folder, create 5 notes: `john-smith.md` (e-commerce client), `sarah-lee.md` (marketing agency client), `mike-chen.md` (potential client), `anna-kowalski.md` (copywriter collaborator), `tom-brown.md` (mentor). Each note should have 2-3 sentences about the person and a Related section at the bottom with `[[wikilinks]]` to relevant projects or other people.

In Projects folder, create 4 notes: `ecommerce-automation-john.md` (linked to `[[john-smith]]` and `[[about-my-business]]`), `sarah-marketing-workflows.md` (linked to `[[sarah-lee]]` and `[[ideal-customer-profile]]`), `openclaw-masterclass-course.md` (linked to `[[about-my-business]]` and `[[anna-kowalski]]`), `personal-brand-website.md` (linked to `[[tom-brown]]` and `[[about-my-business]]`). Each note should have 3-4 sentences about the project and a Related section with `[[wikilinks]]`.

In Business folder, create 2 new notes: `ideal-customer-profile.md` (description of ICP, linked to `[[about-my-business]]`, `[[ecommerce-automation-john]]`, `[[sarah-marketing-workflows]]`), `pricing-and-offers.md` (description of pricing tiers, linked to `[[about-my-business]]` and `[[openclaw-masterclass-course]]`).

In Resources folder, create 3 notes: `ai-agent-best-practices.md` (linked to `[[openclaw-masterclass-course]]`), `prompts-library.md` (linked to `[[ecommerce-automation-john]]` and `[[sarah-marketing-workflows]]`), `openclaw-docs.md` (linked to `[[openclaw-masterclass-course]]` and `[[ai-agent-best-practices]]`).

In Daily folder, create 2 notes: `2026-04-06.md` (today I worked on `[[ecommerce-automation-john]]` and had a call with `[[sarah-lee]]`) and `2026-04-07.md` (today I focused on `[[openclaw-masterclass-course]]` and reviewed notes with `[[anna-kowalski]]`).

In Inbox folder, create 2 notes: `random-ideas.md` (some scratch ideas) and `to-research.md` (linked to `[[ai-agent-best-practices]]`).

After creating all the notes, commit with message 'add demo vault content with wikilinks' and push to GitHub. Then confirm everything is done.

When your agent finishes, run a manual pull in Obsidian (Cmd+P → git pull) and open the **Graph View** from the left ribbon. You'll see all your new notes with lines between them wherever the wikilinks are.

## Colour the graph by folder

To get the colourful look from the lesson intro, click the filter icon on the left edge of the graph view and open the **Groups** section. Click **New group** and add one group per folder using a simple path query. Pick a different colour for each one:

| Query         | Suggested colour |
|---------------|------------------|
| path:Business | Red / orange     |
| path:Projects | Orange / yellow  |

|                |             |
|----------------|-------------|
| path:People    | Light green |
| path:Daily     | Dark green  |
| path:Resources | Turquoise   |
| path:Inbox     | Blue        |

# Five tips for getting the most out of your vault

## Tip 1 — Be patient with the 5-minute sync

The default sync interval is slow on purpose. It keeps both sides in sync without hitting GitHub too often. If you ever need something synced right away, open the command palette with Cmd+P or Ctrl+P and run **git pull** or **git commit-and-sync**. Both of these run straight away, no waiting.

## Tip 2 — Be specific when asking your agent to create notes

Vague requests like 'save this somewhere' give you vague results. Tell your agent which folder to use and what file name you want, and you'll get clean, predictable output every time. Instead of 'remember this', try 'create a note in Resources called client-onboarding-checklist with these sections'.

## Tip 3 — Start small and grow the vault over time

Don't try to move everything into the vault on day one. Start with a few notes about your active projects and a couple of clients you're working with right now, then add more as you see what's missing. After a few weeks of real use, the vault becomes a surprisingly complete picture of your work.

## Tip 4 — Plan for token rotation

Your Personal Access Token expires after the time you picked (90 days by default). Once it expires, GitHub starts rejecting your agent's pushes and pulls with errors that can look confusing if you forgot about it. When that happens, go back to [github.com/settings/tokens](https://github.com/settings/tokens), create a new one with the same repo scope, and send it to your agent the same way you did today. The whole thing takes about a minute.

## Tip 5 — Review what your agent writes

Your agent usually does a great job, but it's still an AI and it sometimes gets things slightly wrong, saves notes in the wrong folder, or writes something that doesn't quite match what you wanted. The good news is that the vault is easy to browse in Obsidian, so a quick look at new notes is usually enough to spot and fix anything that's off. Treat it like reviewing the work of a junior teammate — mostly great, sometimes needs a small correction.

# Troubleshooting

## Clone fails with 'could not read Username for github.com'

This is expected on the first attempt. It means GitHub is rejecting the clone because no credentials were sent. Go to the **Create a Personal Access Token** section and use Prompt 2 to send a clone URL that includes the token.

## Git refuses to commit — 'no author identity configured'

Your agent tried to make a commit before setting a git author. Use Prompt 4 to set **user.name** and **user.email** on the repo. The commit will work right after.

## Obsidian says it's syncing but nothing actually moves

Open the Git plugin settings again and check three things: the **Split timers** toggle is on, all three interval fields (commit, push, pull) are set to 5, and the status indicator in the bottom-right corner of Obsidian isn't showing a red error icon. If it is, click it to see the error message.

## Files appear with weird 'conflict' copies

This happens when both sides edit the same note between sync intervals and git can't merge them automatically. Open the conflict files, pick the version you want to keep, delete the other, and run **Git: Commit-and-sync** manually to clean things up. To avoid this, try not to edit the same note at the same time on your laptop and through your agent.

## Git keeps asking your agent for credentials on every operation

The credential helper isn't saving properly. Send Prompt 3 again and make sure your agent runs the **git config credential.helper store** command from inside the **~/ace-vault** folder, not from the home folder.

## Mac only — files disappear or show cloud icons

iCloud Drive is causing problems. Move the vault to a folder outside iCloud (your home folder is a safe choice), or open System Settings → Apple ID → iCloud → iCloud Drive and turn off Documents sync. Re-clone the vault at its new location if you moved it.

## Local install path — the shorter route

If you installed OpenClaw directly on the same computer where Obsidian lives, the setup is much easier. Your agent and Obsidian share the same filesystem, so your agent can read and write your vault folder directly without needing any sync layer in the middle. You can finish the whole setup with just one prompt.

### Basic local setup

- Install Obsidian on your computer (Step 1 of the main walkthrough)
- Create a folder somewhere — for example `~/Documents/ace-vault`
- Open it as a vault in Obsidian and add the six folders from Step 5
- Add the **about-my-business** note from Step 6
- Skip the GitHub account, the Personal Access Token, and the Git plugin
- Send your agent the local setup prompt below

### Local setup prompt

*My Obsidian vault is at /Users/your-name/Documents/ace-vault (adjust this path to your actual location). Please remember this path permanently and treat this folder as my shared knowledge base. From now on whenever I ask you about my projects, my business, or my notes, search this vault first. When you create new notes or summaries, save them in the appropriate folder inside this vault. The vault has six folders: Inbox, Daily, Projects, Resources, Business, and People. Confirm when you've saved this to memory.*

That's it. Your agent reads and writes the folder directly, and Obsidian picks up the changes right away because everything happens on the same computer. Both the read and write tests from the testing section still work the same way — just skip the git pull and git push steps because there's no remote to sync with.

### Should you add GitHub to a local install anyway?

The short answer is: it's optional, but it's a good idea. Even when your agent runs locally and doesn't need GitHub for syncing, you still get real benefits from adding it later:

- **Free off-site backup.** If your laptop dies, gets stolen, or the drive fails, every note you've written to the vault is still safe on GitHub. You can clone the repo on a new machine and pick up right where you left off.
- **Sync between multiple devices.** If you sometimes work on a second laptop, a desktop at home, or a work machine, GitHub keeps all your devices in sync automatically. Write a note on one computer, it shows up on the others within minutes.
- **Version history.** Every change gets saved as a git commit, so you can look back at earlier versions of any note or recover notes you deleted by mistake. It's like having unlimited undo across your whole vault.

- **Mobile access.** If you ever want to use Obsidian on your phone or tablet, you need a cloud sync in the middle — GitHub is a free way to do that.

If any of that sounds useful, you can follow the main walkthrough above to add GitHub on top of your local setup whenever you're ready. The two approaches work well together — your agent keeps reading the local folder directly as before, and the Obsidian Git plugin handles the sync in the background.

## Recap — what you now have

At this point you've built a full co-working setup between yourself and your agent. Here's what you have:

- Obsidian installed and running on your laptop
- A private GitHub repo called ace-vault acting as the sync bridge
- The repo cloned into Documents and opened as your Obsidian vault
- Six-folder structure: Inbox, Daily, Projects, Resources, Business, People
- Auto-sync every 5 minutes through the Obsidian Git plugin
- Your agent connected to the repo on the VPS with credentials saved for this repo only
- Permanent rules in your agent's memory to pull before reading and push after writing
- A working two-way sync proven by both the read test and the write test

From here on, every note you add makes your agent a bit more useful for your specific situation. The vault grows naturally as you use it for real work, and your agent always has a full picture of whatever matters to you at that moment.